

# Lesson:

# CSS Specificity and !important



# Topics Covered

- Understanding CSS specificity
- How to calculate Specificity order
- Understanding Cascading order
- Introduction to !Important in CSS
- Drawbacks of using !Important

## Understanding CSS specificity

The CSS specificity refers to the level of importance or weight assigned to a particular style rule. It determines which style rule takes precedence when multiple rules target the same HTML element. CSS specificity is calculated based on the combination of different selectors used in a style rule. The higher the specificity of a style rule, the more precedence it has over other rules

The following rules are used to calculate CSS specificity:

### 1. Inline styles:

Inline styles are applied directly to HTML elements using the style attribute. They have the highest specificity because they are applied directly to the element and override other styles.

#### Example of Inline styles

```
Unset
/*Example of inline style*/
<div style="color: red;"> Hello </div>
```

### 2. ID selectors:

ID selectors are used to target elements with specific ID attributes. They have higher specificity than class selectors and element selectors.

#### Example of ID selectors

```
Unset
/* Example of ID selector */
#myId { color: red}
```

### 3. Class selectors:

Class selectors are used to target elements with a specific class attribute. They have lower specificity than ID selectors but higher specificity than element selectors.

#### Example of ID selectors

```
Unset
/* Example of class selector*/
.className{ color:black}
```

#### 4. Element selectors:

Element selectors target elements based on their tag names. They have the lowest specificity and are overridden by ID selectors, class selectors, and inline styles.

#### Example of Inline styles

```
Unset
/* Example of element selector */
h1 { color: purple}
```

## How to calculate Specificity order

CSS specificity is calculated based on the type of selector used and is represented by a four-digit value.

Here is how to calculate specificity -

- Inline style gets a specificity value of 1000 and is always given the highest priority.
- Id selector, for each id selector, 100 is added to the specificity value.
- Class selector, for each class selector, 10 is added to the specificity value.
- Element selector, for each type selector, 1 is added to the specificity value.
- Universal selector is ignored, i.e. 0 is added to the specificity value.

The table below shows some examples on how to calculate specificity values:

| SELECTOR                       | SPECIFICITY VALUE | CALCULATION                           |
|--------------------------------|-------------------|---------------------------------------|
| p                              | 1                 | 1                                     |
| p.test                         | 11                | 1 + 10                                |
| p#demo                         | 101               | 1 + 100                               |
| <p<br>style="color:pink<br>;"> | 1000              | 1000                                  |
| #demo                          | 100               | 100                                   |
| .test                          | 10                | 10                                    |
| p.test1.test2                  | 21                | 1 + 10 + 10                           |
| #navbar<br>p#demo              | 201               | 100 + 1 + 100                         |
| *                              | 0                 | 0 (the universal selector is ignored) |

### Note -

- The selector with the highest specificity value will win and take effect!
- There is one exception to this rule: if you use the **!important** rule, it will even override inline styles.

### Example

#### index.html

```
Unset
<body>
  <h1 class="heading" style="color: green;">CSS
Specificity</h1>
</body>
```

#### style.css

```
Unset
h1 {
  color: red;
}
h1#heading {
  color: yellow;
}
```

### Browser output:

**CSS Specificity**

### From the above code we get three specificities i.e -

1. h1 (type selector) with specificity value of 1
2. h1#heading (type selector with id) with specificity value of  $100 + 1 = 101$
3. Inline styling with the specificity of 1000

Since the third rule with inline styling has the highest specificity value of 1000, this style declaration will be applied.

## Understanding Cascading Order

It refers to the order in which style rules are applied when there are multiple rules targeting the same HTML with the same specificity. The order in which the style rules are defined in the CSS file determines their cascading order.

The class attribute in the HTML can be written in any order as the order of class attributes does not affect the specificity or the cascading order of CSS rules, the specificity of a selector is determined by the selector itself, not the order of classes. Similarly, the cascade or cascading order is determined by the specificity and the order of appearance of CSS rules in the stylesheet.

The later rule in the CSS file takes precedence over the earlier rule when both rules have the same specificity. This means that if two rules with the same specificity conflict, the one defined later in the CSS file will be applied.

## Example

### index.html

```
Unset
<h1 class="heading1 heading2" > Hello World </h1>
```

### style.css

```
Unset
.heading1 {
  color: green;
  text-decoration: underline;
}
/* According to the Cascading order rule - the below rule will
be applied */
.heading2 {
  color: red;
  text-decoration: line-through;
}
```

### Browser output:



## Introduction to !Important in CSS

The “!important” rule is used to give the style rule the highest priority, which means it overrides all other styles applied to the same element.

When the “!important” rule is added to a style property, it will take precedence over any other styles applied to the same element, regardless of where those styles are defined in the CSS file or in the HTML code. This can be useful in situations where a specific style needs to take precedence over all others, such as when debugging a layout issue or when working with third-party CSS libraries.

## Example

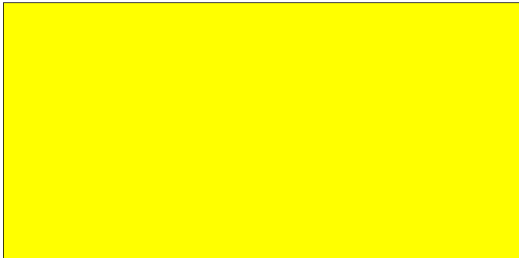
### index.html

```
Unset
<div class="container"></div>
```

### style.css

```
Unset
.container {
  height: 50vh;
  border: 1px solid;
  background-color: yellow !important;
}
/* This will override the background color*/
.container {
  background-color: green;
}
```

### Browser output:



## Drawbacks of using !Important

Some of the drawback point to be considered when using the **!important** CSS rule are as follows.

- It is more difficult to maintain and update styles in the future. Because it takes precedence over other styles.
- It can create specificity conflicts and make it harder to override styles in specific situations
- Debugging issues that arise from the use of “**!important**” can be more difficult, as styles can be overridden unexpectedly and cause layout issues.

### When you have to use and avoid the “!important” CSS rule

When to use –

- Overriding styles from third-party libraries: Sometimes, you may need to override styles from a third-party library that you cannot modify directly. In this case, using !important can be a quick and easy way to override the styles without modifying the original library.

- Debugging layout issues: In some cases, using **!important** can help you quickly identify and fix layout issues that are caused by conflicting styles.
- Applying accessibility styles: Using **!important** can help ensure that accessibility styles are applied consistently across your site, even if other styles conflict with them.

### When to avoid

- Maintenance and scalability: Overuse of the **!important** rule can make it harder to maintain and scale your styles over time, especially when working on larger projects.
- Specificity conflicts: Overuse of the **!important** rule can lead to specificity conflicts, making it harder to override styles in specific situations.
- Unpredictable behaviour: Overuse of the **!important** rule can lead to unpredictable behaviour in your styles, especially when working with larger and more complex stylesheets.

